

Git, conceptually

A workshop handout for when Claude does the typing.

You don't need to memorize commands. In most of your work, you'll *describe what you want* — "save this," "try this on a separate copy," "send it up for review" — and Claude will run the actual git commands for you. This handout gives you the mental model underneath those requests, so you can direct the work with confidence and know what you're looking at when something goes sideways.

The one promise worth internalizing up front: **git is built so you almost never lose work**. Once something is committed, it can be recovered. That single fact is what makes experimentation safe.

While git was developed as software for large teams of developers, it is an immensely useful tool for individuals. Unlike **GitHub** (a website for hosting git repositories online), **git** itself is software that lives on your computer.

THE ONE ANALOGY THAT COVERS MOST OF IT

Think of a writing project with a serious revision history: a manuscript you've been drafting for months.

- The **repository** is the whole project, including every past version — not just what's on screen now.
- A **commit** is a dated, labeled draft you've saved: "*added the methods section*," "*fixed citations*."
- A **branch** is a working copy where you try something risky without touching the clean draft.
- **Merging** is folding a good revision back into the clean draft.
- **GitHub** is the shared cloud copy your collaborators can see and contribute to.
- A **pull request** is handing someone a revision and saying "*review this before it goes into the official version*."

Now hold that picture. Everything below is just the precise version of it.

THE CORE IDEAS

REPOSITORY ("REPO")

Your project folder, plus a complete memory of its history. From the outside it looks like an ordinary folder of files. Hidden inside is a record of every committed version, who changed what, and when.

What Claude does for you: *turns an ordinary folder into a repo ("initialize a git repo here"), or pulls down an existing one from GitHub.*

COMMIT

A saved snapshot of your project at a moment in time, with a short note describing what changed. Commits are the save points in your history. Each one builds on the last, forming a timeline you can revisit.

The note (the "commit message") matters more than users new to git sometimes expect — it's how future-you understands what past-you was doing. Good messages are short and descriptive: "*add reading quiz to week 3*," not "*stuff*."

What Claude does for you: *selects which changes to include (**git add**), then commits (**git commit**) them with a message (**-m**): **git commit -m "add reading quiz to week 3"**. You can always ask it to commit "with a message about X."*

BRANCH

A parallel line of work. You split off from the main version, make changes freely, and the main version stays untouched until you decide to combine them. This is the heart of safe experimentation: a branch is a sandbox.

By convention the primary line is called **main** (sometimes **master** in older projects) — think of it as the trunk, the version you trust.

What Claude does for you: "*make a branch for the new grading feature*" → Claude creates and switches to it. Your **main** stays clean the whole time.

IF YOU'VE EVER MADE A BACKUP FILE, YOU ALREADY UNDERSTAND BRANCHES

When you're deep into a project and something risky is coming — a big restructure, a pivot you're not sure about — the instinct is to duplicate the file first. **syllabus_BACKUP.docx**, **index_before_changes.html**. That instinct is exactly right. A branch is just that instinct made systematic: instead of a duplicate file cluttering your folder, you get a named parallel version that remembers where it split off from the original. If the experiment fails, you can throw the branch away and **main** will never notice. If the branch succeeds, merge it back in cleanly.

The difference worth noting: a backup file is a *static copy*. A branch is a *divergence point* — starting from where it was created. Git tracks the relationship between them, so combining them later (if you want to) is automatic rather than manual.

MERGE

Combining the work from one branch back into another (usually back into **main**). If the two haven't touched the same lines, this is seamless. If they have, git flags a **merge conflict** — it's asking a human (or Claude) to decide which version wins. Conflicts sound scary but are just "*two drafts disagree about this paragraph; please choose*."

What Claude does for you: *performs the merge and, when conflicts arise, walks you through resolving them.*

REMOTE, GITHUB, PUSH, AND PULL

Your repo lives on your computer. A **remote** is a copy that lives elsewhere (almost always on **GitHub**) so that it's backed up and shareable.

■ **Push** — upload your local commits to the remote ("send my work up to a GitHub repo").

■ **Pull** — download commits others have added ("get the latest changes from a GitHub repo").

■ **What Claude does for you:** *pushes and pulls on request. "Push this to GitHub" is a complete instruction.*

PULL REQUEST ("PR")

A proposal to merge one branch into another, wrapped in a review step. Instead of quietly folding your work into **main**, you open a PR on GitHub that says "here's what I changed — take a look before it goes in." Collaborators comment, suggest edits, and approve. Then it gets merged.

This is the social layer of git. It's how teams keep **main** trustworthy: nothing important lands without a second set of eyes.

■ **What Claude does for you:** *"open a pull request for this branch" → Claude pushes the branch and creates the PR with a summary of the changes.*

WORKTREES

Normally you have one branch checked out in one folder at a time — to switch tasks, you switch branches in that same folder. A **worktree** lets you have several branches checked out in several folders *at once*, all sharing the same project history.

Why a newcomer should care: this is how Claude can work on two things in parallel without them colliding. One folder can hold a session building a new feature while another holds a session fixing a bug — neither touches the other's files, but both belong to the same project.

■ **What Claude does for you:** *you can literally say "work on this in a worktree," and Claude sets up an isolated folder for the task. When it's done and clean, the worktree is cleared away automatically. You mostly won't manage these by hand — just know that "Claude is working in a worktree" means "Claude is working in an isolated copy so it can't break your main one."*

HOW THIS MAPS TO TALKING WITH CLAUDE

You'll rarely type git commands. You'll say something like the middle column, and Claude handles the right column:

WHEN YOU WANT TO...	SAY SOMETHING LIKE...	WHAT'S HAPPENING UNDERNEATH
Save your progress	"Commit this with a note about the new syllabus section."	Claude stages changes and writes a commit.
Try something risky	"Make a branch and experiment with the new layout there."	Claude branches off main so the trunk stays safe.
Back up / share your work	"Push this to GitHub."	Claude uploads your commits to the remote.
Get a teammate's latest work	"Pull the latest changes."	Claude downloads new commits from the remote.
Propose finished work for review	"Open a pull request for this branch."	Claude pushes the branch and opens a reviewable PR.
Fold approved work into the trunk	"Merge this into main."	Claude combines the branch into main .
Get back an earlier version	"Revert to before the layout change."	Claude finds the right commit and restores it — but only if a commit was made at that point.
Do two tasks at once	"Work on the bugfix in a separate worktree."	Claude spins up an isolated copy to avoid collisions.

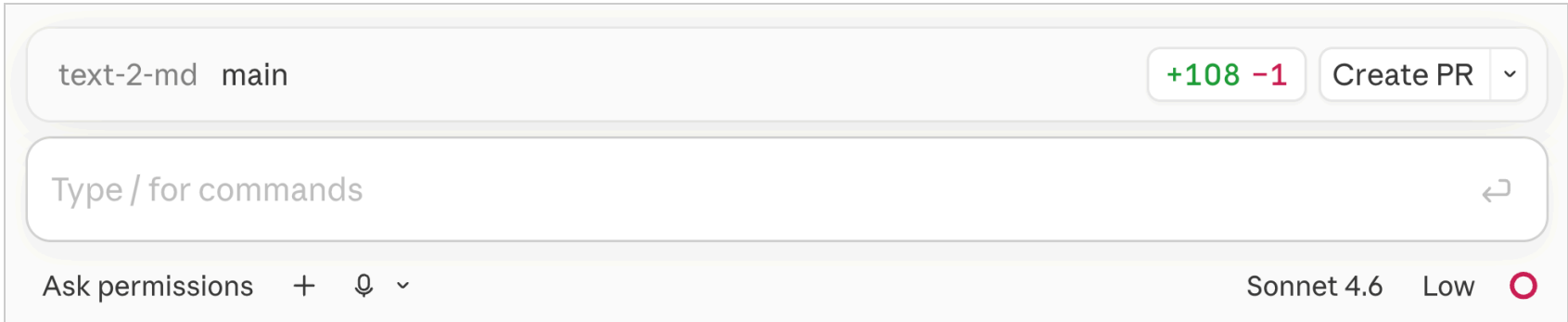
■ The skill you're actually building isn't typing — it's **knowing what to ask for and when**. Commit often (small save points beat giant ones). Branch before anything experimental. Open a PR when you want a human check before it counts.

WHEN CLAUDE ASKS FOR PERMISSION

In a Cowork or Code session, Claude will sometimes pause before running a git command and ask whether to proceed. These aren't warnings that something is wrong. They're Claude applying exactly the instincts described above. Here's when you'll typically see them and what they mean:

- **"Should I commit this?"** → Claude wants to save a snapshot. Say yes if you've reached a stable point, or ask it to wait until you have. It helps to give it a message: *"commit this with a note about the new rubric section."*
- **"I'd like to create a branch here — okay?"** → Claude is about to try something it isn't certain about and wants to protect your main. This is almost always the right call. Say yes.
- **"Push this to GitHub?"** → Claude wants to upload your commits to the remote. "Yes" to back up your work or share it; *"not yet"* if you're mid-experiment and not ready.
- **"Work on this in a worktree?"** → Claude is offering to work in an isolated copy so it can't disturb your current files. You can say yes without fully understanding worktrees — just know it means Claude is being careful.

One thing worth noting: the project name and branch shown in the desktop app always refer to your **session's home project** — where Claude Code was launched from, not necessarily the file you're currently editing:



In the screenshot: **text-2-md** (top left) is the home project; **main** is the active branch. The **+108 -1** shows lines added and removed since the last commit — if that number is climbing and you haven't committed recently, it's worth pausing to save a snapshot. **Create PR** opens a pull request directly from the interface.

You don't need to know the commands to answer the questions above. You just need to know what's at stake — which is what this page is for.

TWO ANXIETIES, DEFUSED

"What if I break something?"

Committed work is recoverable; that's part of what makes git so useful. Experiments live on branches, so your trusted **main** version isn't at risk. The worst case for an experiment gone wrong is throwing the branch away: **main** never noticed.

"There are so many commands."

There are, and you can ignore almost all of them. Understanding the *concepts* on this page is more important for now. The commands are an implementation detail you can delegate.

GLOSSARY AT A GLANCE

Repo A project folder with a full memory of its history.	Git asking a human to choose when two changes disagree.
Commit A labeled save point.	Remote / GitHub The shared cloud copy.
Branch A safe parallel copy for experiments.	Push Upload your commits.
main The trusted primary version (the trunk).	Pull Download others' commits.
Merge Combine a branch's work back in.	Pull request (PR) A reviewable proposal to merge.
Conflict	Worktree Multiple branches open in separate folders at once.

OPTIONAL: PEEKING AT THE TERMINAL

You don't need these commands, but if you ever want to see what Claude is running, the common ones are:

```
git status           # what's changed right now
git add .            # stage everything for the next commit
git commit -m "message"  # save a snapshot with a note
git branch my-feature  # create a branch
git switch my-feature  # move onto a branch
git switch main        # move back to main
git merge my-feature   # fold it back into the current branch
git push             # upload commits to GitHub
git pull             # download new commits
git log --oneline     # the history, one line per commit
git worktree add ../project-bugfix bugfix  # open another branch in its own folder
```

If a command scrolls past and you're not sure what it did, just ask Claude: **"what did that just do, and is anything at risk?"** That question is always fair game.